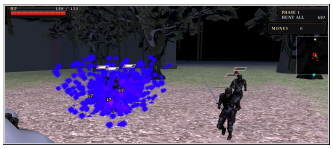
作品紹介（個人制作）

Skybound Slayer

田中 壮

敵を倒して強化し、ボスに挑む3DアクションRPG。基礎を自分で作った作品です。

開発環境	C++ / DxLib / Effekseer（ゲームエンジンなし）
体制・期間	1人 ／ 1〜2か月 ／ 就職活動用に制作


通常戦闘


エフェクト戦闘


ボス戦


ショップ

誰が作ったか

自分

- 企画・設計
- キャラの動きの状態管理（タグ分類）
- 当たり判定
- メモリの解放漏れの修正

AI

- 画像、音楽
- ソースコードのコメント
- エフェクトの再利用の仕組み（EffectPool）
- コードの添削

修正点は、先生とAIにも相談しました。

苦労したことと、直し方

① 動きが増えて、条件が複雑に

最初は `if-else` だけで作成。攻撃や回避を足すたびに、エラーが出やすくなった。

▶

→ 状態ごとに分け、タグで分類

「この状態のときは回避できない」をグループ単位で決め、条件に合わなければ先に処理を打ち切る（ガード）。

結果：エラーが減り、コードも減った。

② 敵が増えると処理落ち・フリーズ

繰り返し遊ぶほど重くなり、最後は強制終了。

▶

→ 解放漏れと解放後の参照を整理

使い終わったものの解放漏れと、解放後のポインタ参照（nullエラー）を見直し、解放の処理を整理。

結果：フリーズしなくなった。

コードの抜粋

自分で作成 — Object3D.h ／ タグの分類

```
enum Tag3D
{
    None3D = 0,
    Tag3D_Player3D = 2000,
    Tag3D_Wall3D   = 2100,
    Tag3D_Enemy3D  = 2200,
    Tag3D_Stage    = 2300,
    Tag3D_Bullet   = 2400,
    Tag3D_Weapon   = 2500,
    ...
};
```

オブジェクトの種類をタグで分け、判定や検索で使う。

AIが作成 — EffectPool.cpp ／ 使い回し

```
void EffectPool::Play(...) {
  for (int i = 0; i < POOL_SIZE; i++) {
    if (!m_Pool[i]->IsActive()) {
      m_Pool[i]->Play(...);
      return;
    }
  }
}
```

空いているエフェクトを探して再利用する（毎回作らない）。

コメントは省略し、一部を省いて掲載しています。

反応・リンク

友人から「ゲームとして一貫性を感じる」という感想をもらいました。

動画：https://youtu.be/drcom4Y--8A ／ GitHub：https://github.com/SoTanaka0409/SkyboundSlayer